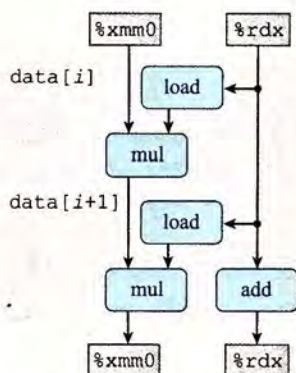


a) 重新排列、简化和抽象图5-18的表示，给出连续迭代之间的数据相关



b) 每次迭代必须顺序地执行两个乘法

图 5-19 将 combine5 的操作抽象成数据流图

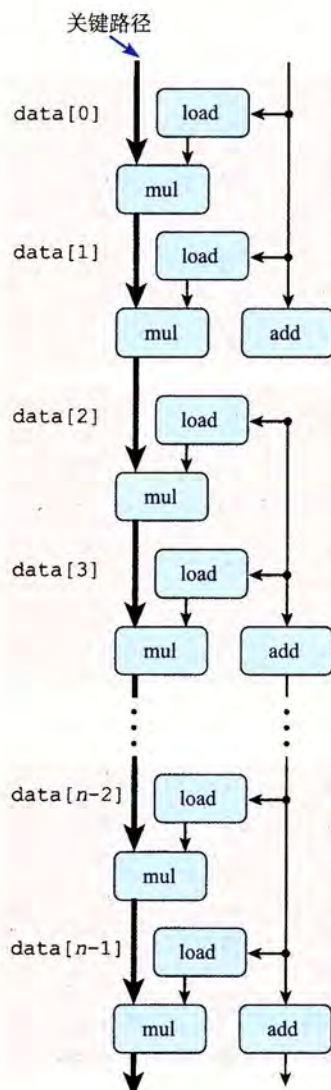


图 5-20 combine5 对一个长度为 n 的向量进行操作的数据流表示。虽然循环展开了 2 次，但是关键路径上还是有 n 个 mul 操作

旁注 让编译器展开循环

编译器可以很容易地执行循环展开。只要优化级别设置得足够高，许多编译器都能例行公事地做到这一点。用优化等级 3 或更高等级调用 GCC，它就会执行循环展开。

5.9 提高并行性

在此，程序的性能是受运算单元的延迟限制的。不过，正如我们表明的，执行加法和乘法的功能单元是完全流水线化的，这意味着它们可以每个时钟周期开始一个新操作，并且有些操作可以被多个功能单元执行。硬件具有以更高速率执行乘法和加法的潜力，但是代码不能利用这种能力，即使是使用循环展开也不能，这是因为我们将累积值放在一个单独的变量 `acc` 中。在前面的计算完成之前，都不能计算 `acc` 的新值。虽然计算 `acc` 新值的功能单元能

够每个时钟周期开始一个新的操作，但是它只会每 L 个周期开始一条新操作，这里 L 是合并操作的延迟。现在我们要考察打破这种顺序相关，得到比延迟界限更好性能的方法。

5.9.1 多个累积变量

对于一个可结合和可交换的合并运算来说，比如说整数加法或乘法，我们可以通过将一组合并运算分割成两个或更多的部分，并在最后合并结果来提高性能。例如， P_n 表示元素 $a_0, a_1, \dots, a_{n-1}$ 的乘积：

$$P_n = \prod_{i=0}^{n-1} a_i$$

假设 n 为偶数，我们还可以把它写成 $P_n = PE_n \times PO_n$ ，这里 PE_n 是索引值为偶数的元素的乘积，而 PO_n 是索引值为奇数的元素的乘积：

$$PE_n = \prod_{i=0}^{n/2-1} a_{2i}$$

$$PO_n = \prod_{i=0}^{n/2-1} a_{2i+1}$$

图 5-21 展示的是使用这种方法的代码。它既使用了两次循环展开，以使每次迭代合并更多的元素，也使用了两路并行，将索引值为偶数的元素累积在变量 $acc0$ 中，而索引值为奇数的元素累积在变量 $acc1$ 中。因此，我们将其称为“2×2 循环展开”。同前面一样，我们还包括了第二个循环，对于向量长度不为 2 的倍数时，这个循环要累积所有剩下的数组元素。然后，我们对 $acc0$ 和 $acc1$ 应用合并运算，计算最终的结果。

比较只做循环展开和既做循环展开同时也使用两路并行这两种方法，我们得到下面的性能：

函数	方法	整数		浮点数	
		+	*	+	*
combine4	在临时变量中累积	1.27	3.01	3.01	5.01
combine5	2×1 展开	1.01	3.01	3.01	5.01
combine6	2×2 展开	0.81	1.51	1.51	2.51
延迟界限		1.00	3.00	3.00	5.00
吞吐量界限		0.50	1.00	1.00	0.50

我们看到所有情况都得到了改进，整数乘、浮点加、浮点乘改进了约 2 倍，而整数加也有所改进。最棒的是，我们打破了由延迟界限设下的限制。处理器不再需要延迟一个加法或乘法操作以待前一个操作完成。

要理解 combine6 的性能，我们从图 5-22 所示的代码和操作序列开始。通过图 5-23

```

1  /* 2 x 2 loop unrolling */
2  void combine6(vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(v);
6      long limit = length-1;
7      data_t *data = get_vec_start(v);
8      data_t acc0 = IDENT;
9      data_t acc1 = IDENT;
10
11     /* Combine 2 elements at a time */
12     for (i = 0; i < limit; i+=2) {
13         acc0 = acc0 OP data[i];
14         acc1 = acc1 OP data[i+1];
15     }
16
17     /* Finish any remaining elements */
18     for (; i < length; i++) {
19         acc0 = acc0 OP data[i];
20     }
21     *dest = acc0 OP acc1;
22 }

```

图 5-21 运用 2×2 循环展开。通过维护多个累积变量，这种方法利用了多个功能单元以及它们的流水线能力

所示的过程，可以推导出一个模板，给出迭代之间的数据相关。同 combine5 一样，这个内循环包括两个 vmulsd 运算，但是这些指令被翻译成读写不同寄存器的 mul 操作，它们之间没有数据相关(图 5-23b)。然后，把这个模板复制 $n/2$ 次(图 5-24)，就是在一个长度为 n 的向量上执行这个函数的模型。可以看到，现在有两条关键路径，一条对应于计算索引为偶数的元素的乘积(程序值 acc0)，另一条对应于计算索引为奇数的元素的乘积(程序值 acc1)。每条关键路径只包含 $n/2$ 个操作，因此导致 CPE 大约为 $5.00/2=2.50$ 。相似的分析可以解释我们观察到的对于不同的数据类型和合并运算的组合，延迟为 L 的操作的 CPE 等于 $L/2$ 。实际上，程序正在利用功能单元的流水线能力，将利用率提高到 2 倍。唯一的例外是整数加。我们已将 CPE 降低到 1.0 以下，但是还是有太多的循环开销，而无法达到理论界限 0.50。

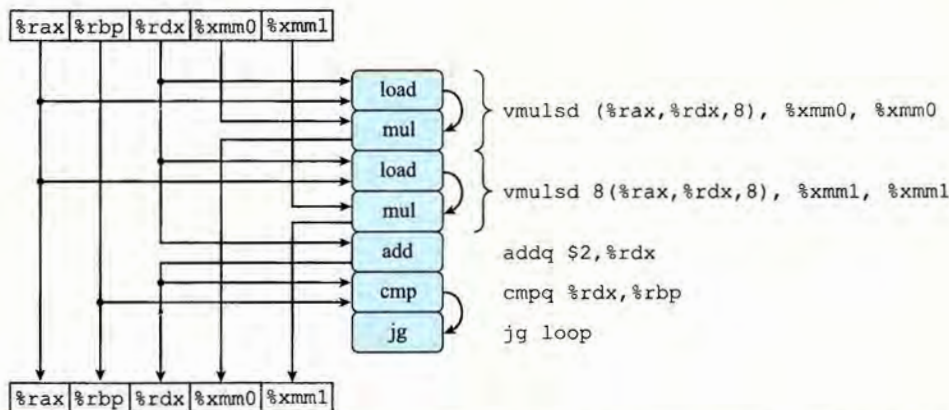
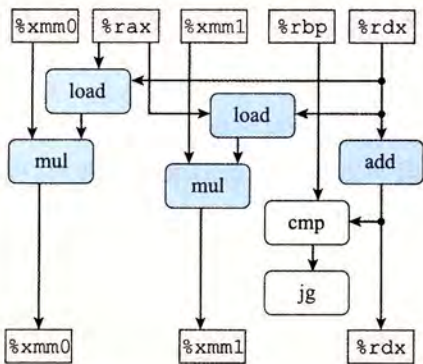


图 5-22 combine6 内循环代码的图形化表示。每次循环有两条 vmulsd 指令，每条指令被翻译成一个 load 和一个 mul 操作

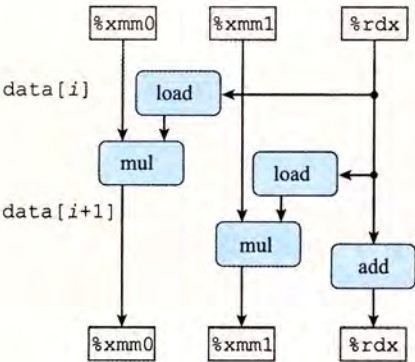
我们可以将多个累积变量变换归纳为将循环展开 k 次，以及并行累积 k 个值，得到 $k \times k$ 循环展开。图 5-25 显示了当数值达到 $k=10$ 时，应用这种变换的效果。可以看到，当 k 值足够大时，程序在所有情况下几乎都能达到吞吐量界限。整数乘和浮点加在 $k=7$ 时达到的 CPE 为 0.54，接近由两个加载单元导致的吞吐量界限 0.50。整数乘和浮点加在 $k \geq 3$ 时达到的 CPE 为 1.01，接近由它们的功能单元设置的吞吐量界限 1.00。浮点乘在 $k \geq 10$ 时达到的 CPE 为 0.51，接近由两个浮点乘法器和两个加载单元设置的吞吐量界限 0.50。值得注意的是，即使乘法是更加复杂的操作，我们的代码在浮点乘上达到的吞吐量几乎是浮点加可以达到的两倍。

通常，只有保持能够执行该操作的所有功能单元的流水线都是满的，程序才能达到这个操作的吞吐量界限。对延迟为 L ，容量为 C 的操作而言，这就要求循环展开因子 $k \geq C \cdot L$ 。比如，浮点乘有 $C=2$ ， $L=5$ ，循环展开因子就必须为 $k \geq 10$ 。浮点加有 $C=1$ ， $L=3$ ，则在 $k \geq 3$ 时达到最大吞吐量。

在执行 $k \times k$ 循环展开变换时，我们必须考虑是否要保留原始函数的功能。在第 2 章已经看到，补码运算是可交换和可结合的，甚至是当溢出时也是如此。因此，对于整数数据类型，在所有可能的情况下，combine6 计算出的结果都和 combine5 计算出的相同。因此，优化编译器潜在地能够将 combine4 中所示的代码首先转换成 combine5 的二路循环展开的版本，然后再通过引入并行性，将之转换成 combine6 的版本。有些编译器可以做这种或与之类似的变换来提高整数数据的性能。



a) 重新排列、简化和抽象图5-22的表示，给出连续迭代之间的数据相关



b) 两个mul操作之间没有相关

图 5-23 将 combine6 的运算抽象成数据流图

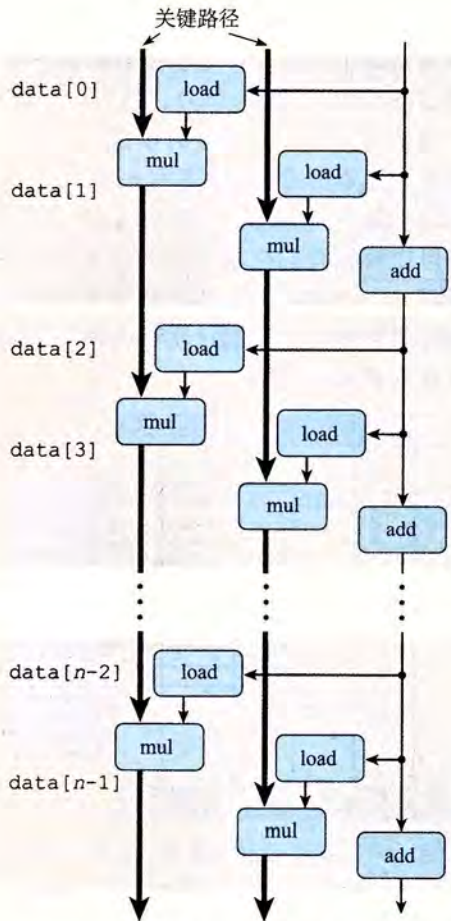


图 5-24 combine6 对一个长度为 n 的向量进行运算的数据流表示。现在有两条关键路径，每条关键路径包含 $n/2$ 个操作

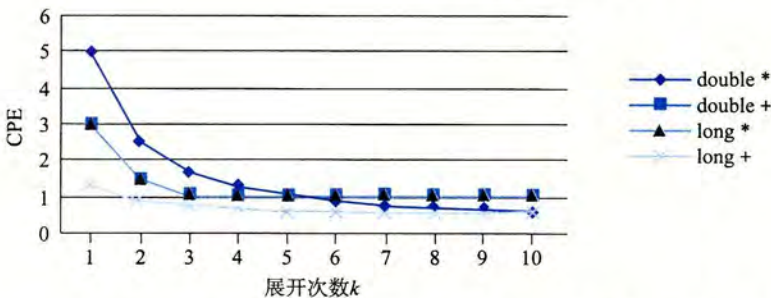


图 5-25 $k \times k$ 循环展开的 CPE 性能。使用这种变换后，所有的 CPE 都有所改进，接近或达到其吞吐量界限

另一方面，浮点乘法和加法不是可结合的。因此，由于四舍五入或溢出，combine5 和 combine6 可能产生不同的结果。例如，假想这样一种情况，所有索引值为偶数的元素都是绝对值非常大的数，而索引值为奇数的元素都非常接近于 0.0。那么，即使最终的乘积 P_n 不会溢出，乘积 PE_n 也可能上溢，或者 PO_n 也可能下溢。不过在大多数现实的程序中，不太可能出现这样的情况。因为大多数物理现象是连续的，所以数值数据也趋向于相

当平滑, 不会出什么问题。即使有不连续的时候, 它们通常也不会导致前面描述的条件那样的周期性模式。按照严格顺序对元素求积的准确性不太可能从根本上比“分成两组独立求积, 然后再将这两个积相乘”更好。对大多数应用程序来说, 使性能翻倍要比冒对奇怪的数据模式产生不同的结果的风险更重要。但是, 程序开发人员应该与潜在的用户协商, 看看是否有特殊的条件, 可能会导致修改后的算法不能接受。大多数编译器并不会尝试对浮点数代码进行这种变换, 因为它们没有办法判断引入这种会改变程序行为的转换所带来的风险, 不论这种改变是多么小。

5.9.2 重新结合变换

现在来探讨另一种打破顺序相关从而使性能提高到延迟界限之外的方法。我们看到过做 $k \times 1$ 循环展开的 `combine5` 没有改变合并向量元素形成和或者乘积中执行的操作。不过, 对代码做很小的改动, 我们可以从根本上改变合并执行的方式, 也极大地提高程序的性能。

图 5-26 给出了一个函数 `combine7`, 它与 `combine5` 的展开代码(图 5-16)的唯一区别在于内循环中元素合并的方式。在 `combine5` 中, 合并是以下面这条语句来实现的

```
12    acc = (acc OP data[i]) OP data[i+1];
```

而在 `combine7` 中, 合并是以这条语句来实现的

```
12    acc = acc OP (data[i] OP data[i+1]);
```

差别仅在于两个括号是如何放置的。我们称之为重新结合变换(reassociation transformation), 因为括号改变了向量元素与累积值 `acc` 的合并顺序, 产生了我们称为“ $2 \times 1a$ ”的循环展开形式。

```
1  /* 2 x 1a loop unrolling */
2  void combine7(vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(v);
6      long limit = length-1;
7      data_t *data = get_vec_start(v);
8      data_t acc = IDENT;
9
10     /* Combine 2 elements at a time */
11     for (i = 0; i < limit; i+=2) {
12         acc = acc OP (data[i] OP data[i+1]);
13     }
14
15     /* Finish any remaining elements */
16     for (; i < length; i++) {
17         acc = acc OP data[i];
18     }
19     *dest = acc;
20 }
```

图 5-26 运用 $2 \times 1a$ 循环展开, 重新结合合并操作。这种方法增加了可以并行执行的操作数量

对于未经训练的人来说, 这两个语句可能看上去本质上是一样的, 但是当我们测量

CPE 的时候，得到令人吃惊的结果：

函数	方法	整数		浮点数	
		+	*	+	*
combine4	累积在临时变量中	1.27	3.01	3.01	5.01
combine5	2×1 展开	1.01	3.01	3.01	5.01
combine6	2×2 展开	0.81	1.51	1.51	2.51
combine7	2×1a 展开	1.01	1.51	1.51	2.51
延迟界限		1.00	3.00	3.00	5.00
吞吐量界限		0.50	1.00	1.00	0.50

整数加的性能几乎与使用 $k \times 1$ 展开的版本 (combine5) 的性能相同，而其他三种情况则与使用并行累积变量的版本 (combine6) 相同，是 $k \times 1$ 扩展的性能的两倍。这些情况已经突破了延迟界限造成的限制。

图 5-27 说明了 combine7 内循环的代码 (对于合并操作为乘法，数据类型为 double 的情况) 是如何被译码成操作，以及由此得到的数据相关。我们看到，来自于 vmovsd 和第一个 vmulsd 指令的 load 操作从内存中加载向量元素 i 和 $i+1$ ，第一个 mul 操作把它们乘起来。然后，第二个 mul 操作把这个结果乘以累积值 acc。图 5-28a 给出了我们如何对图 5-27 的操作进行重新排列、优化和抽象，得到表示一次迭代中数据相关的模板 (图 5-28b)。对于 combine5 和 combine7 的模板，有两个 load 和两个 mul 操作，但是只有一个 mul 操作形成了循环寄存器间的数据相关链。然后，把这个模板复制 $n/2$ 次，给出了 n 个向量元素相乘所执行的计算 (图 5-29)，我们可以看到关键路径上只有 $n/2$ 个操作。每次迭代内的第一个乘法都不需要等待前一次迭代的累积值就可以执行。因此，最小可能的 CPE 减少了 2 倍。

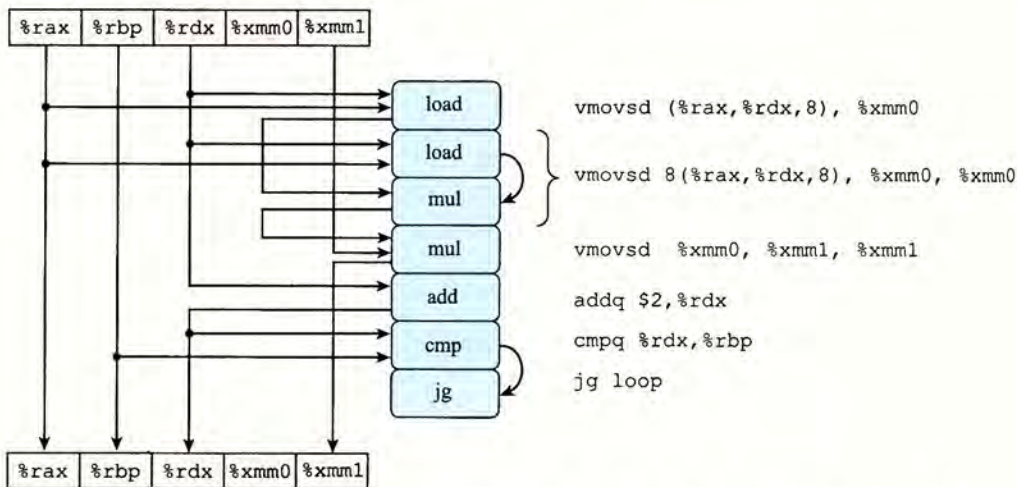
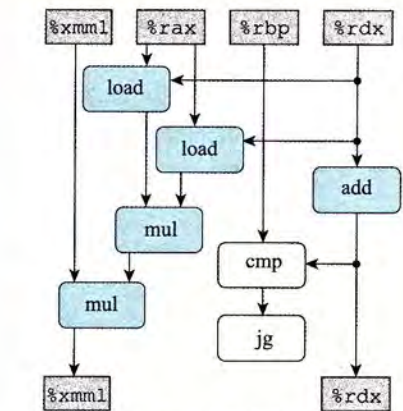
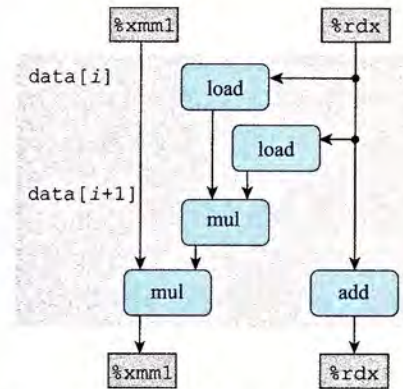


图 5-27 combine7 内循环代码的图形化表示。每次迭代被译码成与 combine5 或 combine6 类似的操作，但是数据相关不同

图 5-30 展示了当数值达到 $k=10$ 时，实现 $k \times 1a$ 循环展开并重新结合变换的效果。可以看到，这种变换带来的性能结果与 $k \times k$ 循环展开中保持 k 个累积变量的结果相似。对所有的情况来说，我们都接近了由功能单元造成的吞吐量界限。



a) 重新排列、简化和抽象图5-27的表示，给出连续迭代之间的数据相关



b) 上面的mul操作让两个二向量元素相乘，而下面的mul操作将前面的结果乘以循环变量acc

图 5-28 将 combine7 的操作抽象成数据流图

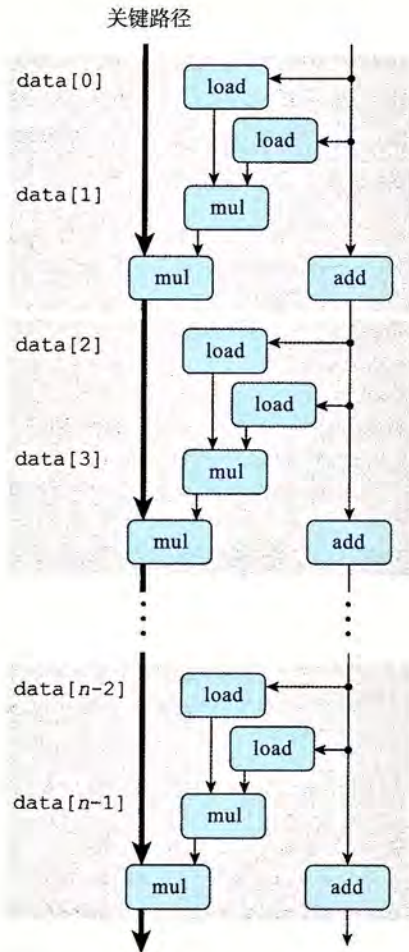


图 5-29 combine7 对一个长度为 n 的向量进行操作的数据流表示。我们只有一条关键路径，它只包含 $n/2$ 个操作

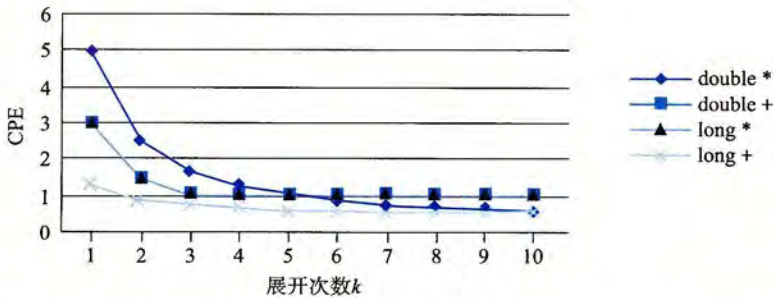


图 5-30 $k \times 1a$ 循环展开的 CPE 性能。在这种变换下，所有的 CPE 都有所改进，几乎达到了它们的吞吐量界限

在执行重新结合变换时，我们又一次改变向量元素合并的顺序。对于整数加法和乘法，这些运算是可结合的，这表示这种重新变换顺序对结果没有影响。对于浮点数情况，必须再次评估这种重新结合是否有可能严重影响结果。我们会说对大多数应用来说，这种差别不重要。

总的来说,重新结合变换能够减少计算中关键路径上操作的数量,通过更好地利用功能单元的流水线能力得到更好的性能。大多数编译器不会尝试对浮点运算做重新结合,因为这些运算不保证是可结合的。当前的 GCC 版本会对整数运算执行重新结合,但不是总有好的效果。通常,我们发现循环展开和并行地累积在多个值中,是提高程序性能的更可靠的方法。

 **练习题 5.8** 考虑下面的计算 n 个双精度数组成的数组乘积的函数。我们 3 次展开这个循环。

```
double aprod(double a[], long n)
{
    long i;
    double x, y, z;
    double r = 1;
    for (i = 0; i < n-2; i+= 3) {
        x = a[i]; y = a[i+1]; z = a[i+2];
        r = r * x * y * z; /* Product computation */
    }
    for (; i < n; i++)
        r *= a[i];
    return r;
}
```

对于标记为 Product computation 的行,可以用括号得到该计算的五种不同的结合,如下所示:

```
r = ((r * x) * y) * z; /* A1 */
r = (r * (x * y)) * z; /* A2 */
r = r * ((x * y) * z); /* A3 */
r = r * (x * (y * z)); /* A4 */
r = (r * x) * (y * z); /* A5 */
```

假设在一台浮点数乘法延迟为 5 个时钟周期的机器上运行这些函数。确定由乘法的数据相关限定的 CPE 的下界。(提示:画出每次迭代如何计算 r 的图形化表示会帮助。)

网络旁注 OPT:SIMD 用向量指令达到更高的并行度

就像在 3.1 节中讲述的,Intel 在 1999 年引入了 SSE 指令,SSE 是“Streaming SIMD Extensions(流 SIMD 扩展)”的缩写,而 SIMD(读作“sim-dee”)是“Single-Instruction, Multiple-Data(单指令多数据)”的缩写。SSE 功能历经几代,最新的版本为高级向量扩展(advanced vector extension)或 AVX。SIMD 执行模型是用单条指令对整个向量数据进行操作。这些向量保存在一组特殊的向量寄存器(vector register)中,名字为 $\%ymm0 \sim \%ymm15$ 。目前的 AVX 向量寄存器长为 32 字节,因此每一个都可以存放 8 个 32 位数或 4 个 64 位数,这些数据既可以是整数也可以是浮点数。AVX 指令可以对这些寄存器执行向量操作,比如并行执行 8 组数值或 4 组数值的加法或乘法。例如,如果 YMM 寄存器 $\%ymm0$ 包含 8 个单精度浮点数,用 $a_0, \dots, a_7$ 表示,而 $\%rcx$ 包含 8 个单精度浮点数的内存地址,用 $b_0, \dots, b_7$ 表示,那么指令

```
vmulps (%rcx), %ymm0, %ymm1
```

会从内存中读出 8 个值,并行地执行 8 个乘法,计算 $a_i \leftarrow a_i \cdot b_i, 0 \leq i \leq 7$,并将得到的

8个乘积保存到向量寄存器`%ymm1`。我们看到，一条指令能够产生对多个数据值的计算，因此称为“SIMD”。

GCC 支持对 C 语言的扩展，能够让程序员在程序中使用向量操作，这些操作能够被编译成 AVX 的向量指令(以及基于早前的 SSE 指令的代码)。这种代码风格比直接用汇编语言写代码要好，因为 GCC 还可以为其他处理器上的向量指令产生代码。

使用 GCC 指令、循环展开和多个累积变量的组合，我们的合并函数能够达到下面的性能：

方法	整数				浮点数			
	int		long		long		int	
	+	*	+	*	+	*	+	*
标量 10×10	0.54	1.01	0.55	1.00	1.01	0.51	1.01	0.52
标量吞吐量界限	0.50	1.00	0.50	1.00	1.00	0.50	1.00	0.50
向量 8×8	0.05	0.24	0.13	1.51	0.12	0.08	0.25	0.16
向量吞吐量界限	0.06	0.12	0.12	—	0.12	0.06	0.25	0.12

上表中，第一组数字对应的是按照 `combine6` 的风格编写的传统标量代码，循环展开因子为 10，并维护 10 个累积变量。第二组数字对应的代码编写形式可以被 GCC 编译成 AVX 向量代码。除了使用向量操作外，这个版本也进行了循环展开，展开因子为 8，并维护 8 个不同的向量累积变量。我们给出了 32 位和 64 位数字的结果，因为向量指令在第一种情况中达到 8 路并行，而在第二种情况中只能达到 4 路并行。

可以看到，向量代码在 32 位的 4 种情况下几乎都获得了 8 倍的提升，对于 64 位来说，在其中的 3 种情况下获得了 4 倍的提升。只有长整数乘法代码在我们尝试将其表示为向量代码时性能不佳。AVX 指令集不包括 64 位整数的并行乘法指令，因此 GCC 无法为此种情况生成向量代码。使用向量指令对合并操作产生了新的吞吐量界限。与标量界限相比，32 位操作的新界限小了 8 倍，64 位操作的新界限小了 4 倍。我们的代码在几种数据类型和操作的组合上接近了这些界限。

5.10 优化合并代码的结果小结

我们极大化对向量元素加或者乘的函数性能的努力获得了成功。下表总结了对于标量代码所获得的结果，没有使用 AVX 向量指令提供的向量并行性：

函数	方法	整数		浮点数	
		+	*	+	*
<code>combine1</code>	抽象-01	10.12	10.12	10.17	11.14
<code>combine6</code>	2×2 循环展开	0.81	1.51	1.51	2.51
	10×10 循环展开	0.55	1.00	1.01	0.52
延迟界限		1.00	3.00	3.00	5.00
吞吐量界限		0.50	1.00	1.00	0.50

使用多项优化技术，我们获得的 CPE 已经接近于 0.50 和 1.00 的吞吐量界限，只受限于功能单元的容量。与原始代码相比提升了 10~20 倍，且使用普通的 C 代码和标准编译器就获得了所有这些改进。重写代码利用较新的 SIMD 指令得到了将近 4 倍或 8 倍的性能提升。比如单精度乘法，CPE 从初值 11.14 降到了 0.06，整体性能提升超过 180 倍。这个例子说明现代处理器具有相当的计算能力，但是我们可能需要按非常程式化的方式来编写程序以便将这些能力诱发出来。